5. Test Approach

5.1 Introduction

The test approach defines the methodology and strategy used to perform load testing on the BlazeDemo Flight Booking Application. The primary goal of the testing process was to evaluate the application's performance, responsiveness, and stability when accessed by multiple virtual users simultaneously.

The load testing activities were performed using Gatling, an open-source performance testing tool. Different business workflows of the application were identified and implemented as separate test scenarios to simulate realistic user behavior.

The testing approach focused on executing critical user transactions, measuring performance metrics, validating server responses, and generating detailed performance reports for analysis.

5.2 Load Testing Strategy

A load testing strategy was adopted to evaluate how the BlazeDemo application performs under expected user load conditions.

The testing process involved simulating multiple virtual users performing flight booking activities simultaneously. Each user followed a predefined business workflow representing actual user interactions with the application.

The strategy included the following activities:

Identification of Critical Business Transactions

The most important functionalities of the application were identified and selected for testing:

- Flight Search
- Flight Selection
- Flight Booking

These functionalities represent the core business operations of the BlazeDemo application.

Scenario-Based Testing

Instead of testing the entire application in a single execution, the workflow was divided into separate scenarios:

Scenario 1 – Search Flights

Users search for available flights by selecting departure and destination cities.

Scenario 2 – Choose Flight

Users search for flights and select a flight from the available options.

Scenario 3 – Complete Booking

Users perform the complete booking process from flight search to booking confirmation.

This approach helped evaluate the performance of each business transaction independently.

Gradual User Load Injection

A ramp-up strategy was used during execution.

Virtual users were gradually introduced into the system over a period of 30 seconds instead of starting all users simultaneously. This approach simulates real-world traffic patterns and helps observe system behavior under increasing load conditions.

Example:

- Total Users: 20 Virtual Users
- Ramp-Up Duration: 30 Seconds

This configuration allows the system to experience a progressive increase in traffic during execution.

5.3 User Simulation

User simulation is the process of creating virtual users that mimic real user behavior.

In this project, Gatling Virtual Users (VUs) were used to perform application transactions automatically.

Each virtual user executed the same workflow that a real user would perform while booking a flight.

Simulated User Activities

The following activities were simulated:

Flight Search Activity

- Open BlazeDemo Home Page
- Select Departure City
- Select Destination City
- Click Find Flights

Flight Selection Activity

- Search Available Flights
- Select a Flight
- Proceed to Booking Page

Complete Booking Activity

- Search Flights
- Choose Flight
- Enter Passenger Information
- Submit Booking
- Verify Confirmation Page

Virtual User Configuration

The user load was configured using Gatling's ramp-up model:

```
rampUsers(20)
.during(Duration.ofSeconds(30))
```

This configuration gradually injects 20 virtual users over a 30-second period.

Think Time Simulation

Pause statements were included between requests to simulate realistic user behavior.

Example:

```
.pause(2)
```

These pauses represent the time a real user spends reading a page or making a decision before proceeding to the next step.

By introducing think time, the simulation becomes more realistic and better reflects actual application usage.

5.4 Request Validation

Request validation ensures that the application responds correctly during test execution.

Every HTTP request executed by Gatling was validated using response checks.

The primary validation performed in this project was HTTP status code verification.

Example:

```
.check(status().is(200))
```

A status code of 200 indicates that the request was processed successfully by the server.

Validation Objectives

The validation process was performed to:

- Verify successful request execution.
- Ensure application availability.
- Detect failed transactions.
- Measure successful request percentage.
- Confirm proper workflow execution.

Validation During Scenario Execution

For every scenario, Gatling validated:

Scenario 1

- Home Page Response
- Find Flights Response

Scenario 2

- Home Page Response
- Flight Search Response
- Choose Flight Response

Scenario 3

- Home Page Response
- Flight Search Response
- Choose Flight Response
- Purchase Flight Response
- Confirmation Page Response

Success Criteria

The following criteria were used to determine successful execution:

- HTTP Status Code = 200
- No Request Failures
- Successful Completion of User Workflow
- Stable Application Response

The test execution results showed a 100% success rate with no failed requests, indicating that the BlazeDemo application successfully processed all simulated user transactions.

5.5 Summary

The test approach adopted for this project combined scenario-based load testing, virtual user simulation, and request validation techniques. Critical business workflows were executed using Gatling virtual users, and server responses were validated to ensure successful transaction processing.

This approach provided reliable performance data and enabled comprehensive analysis of the BlazeDemo Flight Booking Application under simulated load conditions.

.